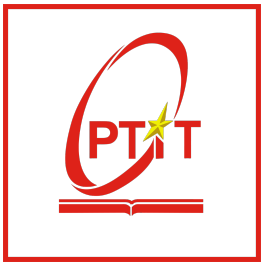
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



TUTORIAL PHÁT TRIỂN HỆ ĐA TÁC TỬ

MÔN HỌC: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

PYTHON, PYTORCH VÀ KIỂM CHỨNG HỆ THỐNG

Giảng viên hướng dẫn	:	PGS.TS. Trần Đình Quế
Lớp	:	D23CTPM01
Nhóm học phần	:	CT01
Sinh viên thực hiện	:	Nguyễn Nam Hải
Mã sinh viên	:	B23DCCN277

Hà Nội – Năm 2026

MỤC LỤC

Danh mục thuật ngữ **iv**

1 Mục tiêu, lộ trình và bản đồ thực hành **1**
1.1 Chuẩn bị môi trường và chạy nhanh 1

2 Unit 1 — Từ mô hình AI đến hệ đa tác tử **1**

3 Unit 2 — Chọn bài toán và xác định lý do dùng MAS **2**

4 Unit 3 — PEAS, tính hợp lý và đặc tả nhiệm vụ **2**

5 Unit 4 — Môi trường, mục tiêu và kiến trúc hybrid **3**

6 Unit 5 — Kiến trúc giao tiếp và sơ đồ vận hành **4**

7 Unit 6 — Quy trình kỹ thuật từ yêu cầu đến kiểm thử **4**

8 Unit 7 — Python cơ bản và hợp đồng đầu vào **5**

9 Unit 8 — Catalog và recommendation **5**

10 Unit 9 — Coordinator và bất biến đầu ra **5**

11 Unit 10 — PyTorch: từ mạng chưa train đến classifier thật **6**
11.1 Ngưỡng tin cậy, từ chối và lỗi ngoài miền 6

12 Unit 11 — Timeout, fallback và giám sát **7**

13 Unit 12 — Case study thương mại điện tử **7**

14 Unit 13 — Case study du lịch **8**

15 Unit 14 — Case study hỗ trợ hành chính y tế **8**

16 Unit 15 — So sánh ba miền và quyền sở hữu ràng buộc **8**

17 Unit 16 — Thiết kế benchmark có đối chứng **9**

18 Unit 17 — Hoàn thành năm bài thực hành **9**
18.1 Exercise 1: Đặc tả tác tử và PEAS 9
18.2 Exercise 2: Mở rộng shopping system 9
18.3 Exercise 3: Huấn luyện intent classifier 10
18.4 Exercise 4: Failure injection 10
18.5 Exercise 5: So sánh kiến trúc 10

19 Unit 18 — Kiểm thử, rubric và lỗi thường gặp **11**

20 Unit 19 — Kiến trúc tham chiếu và hướng phát triển **11**

21 Unit 20 — Tổng kết, demo và câu hỏi bảo vệ **11**

22 Kết quả thực hành và bằng chứng chạy **12**
22.1 Kiến trúc B0–B3 12
22.2 Chỉ số classifier trên held-out test 12
22.3 Ngoài miền: những lỗi cần giữ lại để phân tích 12
22.4 Kết quả failure injection và test 13
22.5 Mã thực thi cần đọc khi bảo vệ 14

Tài liệu tham khảo **16**

DANH MỤC HÌNH VẼ

1 Kiến trúc cục bộ: các quyết định có vai trò riêng, kết quả đi qua coordinator. 4
2 Loss train/validation và confusion matrix trên 108 câu test tổng hợp. 13

DANH MỤC THUẬT NGỮ

MAS	Multi-Agent System – Hệ đa tác tử
LLM	Large Language Model – Mô hình ngôn ngữ lớn
RL / MARL	Reinforcement Learning / Multi-Agent RL
MDP / POMDP	Quá trình quyết định Markov / quan sát không đầy đủ
TD / Q	Sai phân thời gian / Giá trị hành động
DP	Dynamic Programming – Quy hoạch động
OOD	Out-of-distribution – Ngoài phân phối/ngoài miền
PEAS	Performance, Environment, Actuators, Sensors
DAG	Directed Acyclic Graph – Đồ thị có hướng không chu trình
SD / CI	Độ lệch chuẩn mẫu / Khoảng tin cậy
P50 / P95	Phân vị 50 / 95 của thời gian phản hồi
TTL	Time to Live – Giới hạn độ mới của dữ liệu

Quy ước đọc báo cáo

Dữ liệu thực hành là tổng hợp hoặc mô phỏng. Số đo được lấy từ mã đã chạy và lưu kèm; phép tính lý thuyết và đề xuất chưa thực hiện được gọi rõ. Thông tin bìa, logo và phong cách trình bày kế thừa báo cáo trước. Ngày hoàn thiện: 28/10/2026.

1 Mục tiêu, lộ trình và bản đồ thực hành

Tutorial này phát triển đầy đủ 20 chủ đề trong slide *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*, đồng thời hoàn thành năm bài thực hành ở Unit 17. Bản mở rộng thêm hợp đồng thông điệp, timeout thực sự, kiểm tra độ mới tồn kho, dữ liệu huấn luyện có chia nhóm, kiểm thử tự động, ablation và bảng đo thực nghiệm. [1]

Kết quả cuối là một trợ lý mua hàng cục bộ có CatalogAgent, RecommendationAgent, InventoryAgent, PolicyAgent và Coordinator; một bộ định tuyến intent bằng PyTorch đã train; một baseline một thành phần; và bộ ca lỗi dùng để kiểm tra phối hợp. Dữ liệu catalog, kho và câu yêu cầu đều **tổng hợp phục vụ học tập**. Không có thanh toán, đặt mua, dữ liệu bệnh nhân hay kết nối dịch vụ ngoài.

Lộ trình đề xuất gồm ba buổi: buổi 1 học khái niệm và chạy luồng mua hàng; buổi 2 đọc hợp đồng, thêm kiểm chứng và thử lỗi; buổi 3 train classifier, chạy benchmark và phân tích kết quả. Người học nên chạy ví dụ nhỏ trước rồi mới đọc các mở rộng. Các lệnh bên dưới chạy từ thư mục `solution/`; script tự tìm thư mục kết quả theo vị trí tệp nên cũng có thể gọi bằng đường dẫn tuyệt đối.

1.1 Chuẩn bị môi trường và chạy nhanh

Mã đã được kiểm tra bằng Python 3.9.6, NumPy và PyTorch CPU trong máy hiện tại; với môi trường mới nên dùng Python 3.10 trở lên có bản PyTorch phù hợp. Hệ MAS cơ bản chỉ dùng thư viện chuẩn; classifier và đồ thị cần các gói ghi trong `requirements.txt`. Các phiên bản thực tế được lưu trong JSON và tệp môi trường, giúp người đọc phân biệt môi trường đã thử với khuyến nghị cài đặt.

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 tutorial/slide_baseline.py
python3 tutorial/mas_system.py
python3 tutorial/train_intent.py
python3 tutorial/evaluate_mas.py
python3 -m unittest discover -s tutorial -p 'test_*.py'
```

Các bước huấn luyện và đánh giá được tách tệp. Nếu chỉ muốn xem demo MAS, chưa cần train classifier. Nếu muốn chạy B2 hoặc kiểm thử liên quan router, cần tạo checkpoint trước, hoặc dùng checkpoint đã kèm trong `results/tutorial/`. Khi dùng Jupyter, mở notebook tutorial, chọn kernel cùng môi trường rồi Run All; đầu trang notebook ghi rõ phạm vi thực thi.

2 Unit 1 — Từ mô hình AI đến hệ đa tác tử

Một agent tiếp nhận thông tin, lựa chọn hành động và chịu trách nhiệm một phần mục tiêu. MAS có nhiều thành viên với quyết định tương tác qua môi trường hoặc thông điệp. Một nhóm hàm thuần túy không tự động chứng minh cần MAS; phải có ranh giới nhiệm vụ có

ý nghĩa, chính sách hoặc tài nguyên riêng và cơ chế phối hợp có thể quan sát.

Trong ví dụ mua hàng, mô hình xếp hạng chỉ trả điểm. Hệ thông minh còn cần truy hồi đúng sản phẩm, kiểm tra ngân sách, xác nhận kho và phản hồi khi không đủ bằng chứng. Chia thành agent giúp xác định ai sở hữu mỗi quyết định. Mã chạy chung một tiến trình; "đa tác tử" ở đây nói về trách nhiệm và chính sách, không phải nhiều máy hay nhiều LLM.

Thực hành: chạy `mas_system.py`, quan sát đầu ra của kịch bản normal, timeout, malformed, stale và forged. Với normal, danh sách là p2 giá 25 và p4 giá 55. Đây là kết quả của lọc dữ liệu, xếp hạng và kiểm chứng kho, không có suy luận ngôn ngữ được giả lập. Ghi lại thành phần nào tạo ra từng trường trong kết quả.

Câu hỏi tự kiểm tra: nếu thay tất cả agent bằng một hàm lớn nhưng giữ hành vi, lợi ích và chi phí thiết kế thay đổi thế nào? Câu trả lời cần nói đến khả năng thay thế thành phần, kiểm thử, logging và overhead, thay vì khẳng định số lượng agent quyết định trí thông minh.

3 Unit 2 — Chọn bài toán và xác định lý do dùng MAS

Các miền e-commerce, tourism và healthcare support đều có thể phân rã thành chuyên môn, nhưng yêu cầu dữ liệu và mức rủi ro khác nhau. Mua hàng cần catalog, giá và tồn kho; lịch trình cần giờ mở cửa, thời gian đi và chi phí; hỗ trợ hành chính y tế cần nội dung đã duyệt và lịch hẹn. Cùng một sơ đồ agent không có nghĩa dùng chung tiêu chí nghiệm thu.

Tutorial chọn e-commerce vì có thể tạo một môi trường nhỏ với ground truth rõ. Mỗi sản phẩm có ID, giá, category và tags; kho có số lượng theo ID. Điều kiện thành công là trả đúng danh sách hợp lệ hoặc trả đúng trạng thái không thể xác minh. Đây là lựa chọn giúp kiểm chứng đầu cuối mà không cần API key.

Chỉ nên tách agent nếu ranh giới giúp quyết định, phân quyền hoặc thử nghiệm. CatalogAgent và InventoryAgent dùng hai loại dữ kiện khác nhau; PolicyAgent kiểm tra điều kiện bắt buộc; RecommendationAgent có thể thay bằng learned ranker sau này. Với một catalog năm sản phẩm, MAS không có lý do để được giả định nhanh hơn baseline đơn; benchmark sẽ đo thay vì suy đoán.

4 Unit 3 — PEAS, tính hợp lý và đặc tả nhiệm vụ

PEAS gồm Performance, Environment, Actuators và Sensors. Với trợ lý mua hàng: Performance là đúng ràng buộc, truy hồi đúng và không khẳng định tồn kho cũ; Environment là catalog và kho tổng hợp; Actuators là trả danh sách hoặc trạng thái lỗi; Sensors là truy vấn, ngân sách và kết quả agent. Không có actuator thanh toán.

Tính hợp lý là chọn hành động tốt theo thông tin và nguồn lực hiện có. Khi kho timeout, "không xác minh được" là kết quả hợp lý dù không đưa ra khuyến nghị hoàn chỉnh. Tạo một câu trả lời tự tin không làm nhiệm vụ thành công nếu thiếu dữ kiện bắt buộc. Do đó success phải bao gồm cả những ca từ chối hoặc fallback đúng.

Thành viên	Đầu vào	Đầu ra	Quyền	Tiêu chí kiểm tra
Catalog	Query	Danh sách ID	Đọc catalog	Tag khớp, query rỗng không trả toàn bộ
Recommender	Candidate ID, budget	Tối đa 3 ID	Đọc thuộc tính	Chỉ candidate hợp lệ, giá không vượt budget
Inventory	ID	Stock và tuổi dữ liệu	Đọc kho	Có đủ ID, stock không âm, dữ liệu mới
Policy	ID, candidate, budget	ID đã xác nhận	Chỉ kiểm chứng	Không cho ID lạ hoặc vượt giá
Coordinator	Yêu cầu	Status và items	Điều phối	Envelope đúng, deadline, đầu ra được kiểm

Mỗi hàng là một hợp đồng thiết kế. Khi một agent xử lý thêm trách nhiệm ngoài hàng của mình, nên giải thích vì sao và bổ sung test. Ví dụ recommender có thể lọc budget để tiết kiệm, nhưng PolicyAgent vẫn kiểm lại vì output thành phần có thể sai. Lặp kiểm tra ở ranh giới tin cậy là chủ đích, không phải hai định nghĩa nghiệp vụ khác nhau.

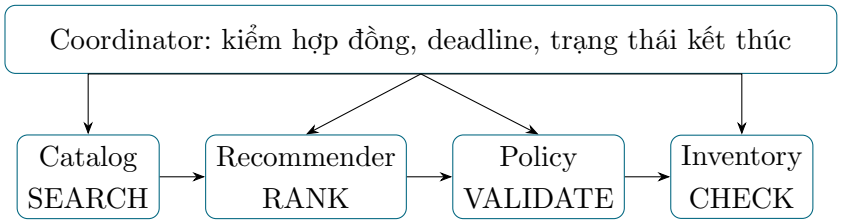
5 Unit 4 — Môi trường, mục tiêu và kiến trúc hybrid

Môi trường catalog nhỏ gần như tĩnh trong một lần gọi, nhưng dữ liệu kho có tuổi và có thể trở nên cũ. Một đề xuất dựa trên dữ liệu vừa đọc chỉ có hiệu lực tại thời điểm đó. Dữ liệu cũ một giờ được tạo trong kịch bản stale nhằm buộc hệ thống phân biệt "có số lượng trong bộ nhớ" với "đã xác minh hiện tại".

Kiến trúc hybrid dùng luật cho ràng buộc cứng và mô hình học cho quyết định dự đoán. Trong tutorial, intent classifier có thể chọn luồng, còn budget và provenance không giao cho mạng tự quyết. Một lỗi phân loại cần được ghi là lỗi định tuyến; một sản phẩm vượt budget cần bị chặn cho dù score mô hình cao đến đâu.

Khi mở rộng, có thể dùng BDI để tổ chức niềm tin, mục tiêu và ý định, hoặc utility để cân bằng sở thích mềm. Tuy nhiên, bất biến như "không trả ID không tồn tại" vẫn cần kiểm chứng trực tiếp. Không nên biến điều kiện bắt buộc thành một trọng số nhỏ trong score rồi hy vọng mô hình tự ưu tiên.

6 Unit 5 — Kiến trúc giao tiếp và sơ đồ vận hành



Hình 1: Kiến trúc cục bộ: các quyết định có vai trò riêng, kết quả đi qua coordinator.

Thông điệp có task ID, sender, receiver, kind và payload. Task ID liên kết các bước thuộc cùng yêu cầu; kind xác định loại thao tác; payload chứa dữ liệu có cấu trúc. Timestamp cho độ mới được mô hình hóa bằng tuổi dữ liệu kho, tránh lẫn với thời gian đo latency. Trace riêng lưu thời lượng thực thi từng lần gọi.

```
@dataclass(frozen=True)
class Message:
    task_id: str
    sender: str
    receiver: str
    kind: str
    payload: dict[str, Any]
```

Mỗi agent xác nhận người nhận và loại yêu cầu; coordinator kiểm sender, receiver, task ID và loại phản hồi. Cùng một payload nhưng task ID sai bị từ chối vì có thể là kết quả của công việc khác. Hệ hiện tại là in-process nên chưa có chữ ký hay xác thực mạng; schema validation không nên được mô tả thành cơ chế bảo mật phân tán hoàn chỉnh.

7 Unit 6 — Quy trình kỹ thuật từ yêu cầu đến kiểm thử

Bắt đầu bằng câu người dùng: “tìm đồ portable không quá 100”. Chuyển thành điều kiện kiểm chứng: query không rỗng, chỉ trả sản phẩm từ catalog, giá không quá 100, stock dương, dữ liệu kho không quá 60 giây. Xác định trạng thái dừng cho không kết quả và lỗi. Sau đó mới phân vai và viết message.

Bản end-to-end tối thiểu cần chạy được trước khi thêm ML. Nó cung cấp nơi kiểm tra lỗi đầu vào, lọc dữ liệu và tính đúng của logic. Khi thêm classifier, giữ nguyên bộ test để phát hiện mô hình mới làm hỏng ràng buộc cũ. Tách các thay đổi giúp biết lỗi đến từ orchestration hay learning.

Tệp `mas_traces.json` là bằng chứng hữu ích khi debug: mỗi case có kết quả cùng chuỗi lời gọi. Nếu kết quả trống, kiểm tra đó là no_results, timeout, availability_unverified hay contract_error. Các trạng thái này có ý nghĩa khác nhau với người dùng và không nên bị gom thành “AI không trả lời”.

8 Unit 7 — Python cơ bản và hợp đồng đầu vào

Catalog, kho và sản phẩm được lưu trong bộ nhớ để ví dụ dễ chạy. `Product` và `Message` dùng dataclass đóng băng, giúp hạn chế sửa vô ý ở cấu trúc ngoài. Lưu ý frozen dataclass không tự làm các dict lồng bên trong bất biến; code tránh sửa payload sau khi gửi và không quảng cáo cấu trúc này là deep immutable.

Coordinator kiểm query là chuỗi, budget là số hữu hạn và không âm. Các ca NaN cần được chặn vì so sánh với NaN có hành vi dễ gây lọt ràng buộc. Tệp test có cả budget âm và NaN. Một kiểm tra kiểu chung chung như "có thể chuyển thành float" chưa đủ để xác nhận ý nghĩa nghiệp vụ.

Thực hành sửa lỗi: thay query bằng chuỗi rỗng và quan sát no_results; thay budget bằng -1 và NaN để quan sát invalid_input. Sau đó đọc `Message.validate` và `Coordinator.call`. Chỉ mở rộng kiểu dữ liệu được chấp nhận khi có lý do và test phù hợp.

9 Unit 8 — Catalog và recommendation

CatalogAgent tìm chuỗi truy vấn trong tên, category hoặc tags của sản phẩm. Truy vấn được strip và chuyển lower. Query rỗng không được xem là khớp mọi chuỗi. Cách tìm này minh bạch, nhưng không hỗ trợ lỗi chính tả, đồng nghĩa, truy vấn nhiều điều kiện hoặc ngữ nghĩa tiếng Việt; đó là giới hạn có chủ đích của tầng retrieval.

RecommendationAgent lọc candidate có trong catalog và giá không quá budget, sắp theo giá tăng rồi lấy tối đa ba sản phẩm. Nó không phải một recommender cá nhân hóa được học từ người dùng. Tên "recommendation" mô tả chức năng đưa gợi ý, không hàm ý đã train mô hình xếp hạng.

Với query portable và budget 100, laptop 850 và tablet 150 bị loại; mouse 25 và backpack 55 còn hợp lệ. Với query sports, giày có giá 70 nhưng stock 0 nên danh sách cuối trống. Hai ca này phân biệt lọc theo giá và xác minh tồn kho, tránh nhầm retrieval có kết quả với nhiệm vụ đã hoàn tất.

10 Unit 9 — Coordinator và bất biến đầu ra

Coordinator thực hiện SEARCH → RANK → VALIDATE → CHECK. Kết quả được kiểm ở mỗi ranh giới. Một message đúng kiểu nhưng sai task ID vẫn thất bại. PolicyAgent đối chiếu cả catalog, candidate ban đầu và budget, giúp chặn một ranker đưa vào ID tự tạo hoặc sản phẩm ngoài truy vấn.

Đầu ra cuối có `status` và `items`; từng item có ID, giá lấy từ catalog và availability. Chỉ trạng thái đã xác minh mới có item được gắn verified. Nếu kho cũ hoặc lỗi, hệ không tái sử dụng một kết luận availability từ bước trước. Đây là tiêu chí cụ thể có thể kiểm bằng failure injection.

Một vấn đề chưa được giải quyết trong prototype là race condition giữa kiểm kho và mua thật. Ở đây không có thao tác mua nên kết quả chỉ là snapshot gợi ý. Nếu thêm checkout,

cần reservation, transaction, idempotency và kiểm tra lại trước xác nhận. Không thể coi việc đã CHECK là đã khóa được hàng.

11 Unit 10 — PyTorch: từ mạng chưa train đến classifier thật

Slide minh họa một mạng nhận hai đặc trưng số nhưng chưa huấn luyện. Bản mở rộng thay bằng dữ liệu câu tiếng Việt có nhãn và bag-of-words. Quy trình chuẩn hóa bỏ dấu, chuyển chữ thường và lấy token a–z; vocabulary chỉ xây từ train. Vector đầu vào có 79 chiều trong lần chạy hiện tại, mạng gồm Linear → ReLU → Linear với hidden size 32 và ba logits đầu ra.

Dữ liệu gồm 432 câu: ba lớp, mỗi lớp tám họ động từ, sáu cụm đối tượng và ba hậu tố. Chia theo họ động từ: năm họ train, một validation và hai test trong mỗi lớp; tương ứng 270/54/108 câu. Không có câu trùng chính xác giữa các split và group ID không giao nhau. Tuy vậy, từ chỉ đối tượng vẫn chia sẻ giữa split, nên đây là kiểm tra tổng hợp tương đối dễ.

Hàm mất mát CrossEntropyLoss nhận logits và nhãn số nguyên; không softmax trước loss. Adam dùng learning rate 0.02, 120 epoch, seed 42 và một CPU thread. Checkpoint được chọn theo validation loss thấp nhất; test chỉ dùng để báo cáo cuối. Tài liệu PyTorch chính thức xác nhận giao diện logits/target này. [10]

```
model.train()
optimizer.zero_grad()
logits = model(train_x)
loss = criterion(logits, train_y)
loss.backward()
optimizer.step()
model.eval()
with torch.inference_mode():
    probabilities = model(test_x).softmax(dim=1)
```

Classifier đạt accuracy và macro-F1 bằng 1 trên 108 câu test, mỗi lớp 36 câu. Đây là số đo thật trên tập tổng hợp, không chứng minh hiểu ngôn ngữ rộng. Không có người dùng thật hoặc dữ liệu bệnh viện trong tập. Mục tiêu của bài thực hành là thực hiện đúng vòng train–validation–test và quan sát giới hạn ngoài phân phối.

11.1 Ngưỡng tin cậy, từ chối và lỗi ngoài miền

Ngưỡng được chọn trên validation để giữ coverage cao với không có lỗi quan sát được; kết quả là 0.5. Router còn yêu cầu câu có ít nhất một token thuộc lexicon miền. Hai lớp kiểm tra này vẫn không bảo đảm phát hiện OOD. Softmax là độ tự tin nội bộ của mô hình, không phải xác suất được hiệu chỉnh rằng yêu cầu thuộc miền.

Sáu câu OOD minh họa được đánh giá sau chọn ngưỡng. Bốn câu được trả clarify, nhưng "thời tiết hôm nay thế nào" bị route sang ecommerce và "giải phương trình" sang tourism. Nguyên nhân có thể liên quan token trùng và độ tự tin cao ngoài miền; chưa có ablation đủ để kết luận định lượng vai trò từng token. Báo cáo giữ nguyên các lỗi để cho thấy accuracy test tổng hợp chưa đủ.

Một cải tiến tiếp theo là tạo tập development OOD và test OOD độc lập, định nghĩa reject class hoặc huấn luyện phát hiện ngoài miền, rồi đo false acceptance và coverage–risk. Không được sửa lexicon theo đúng sáu câu đã xem rồi báo rằng hệ đã tổng quát hóa. Nếu dùng temperature scaling, nhiệt độ cũng phải chọn bằng validation riêng.

12 Unit 11 — Timeout, fallback và giám sát

`asyncio.wait_for` áp deadline thực sự lên coroutine gọi agent. Inventory chậm được mô phỏng bằng sleep 50 ms, trong khi deadline mặc định là 10 ms. Khi quá hạn, lời gọi bị hủy và coordinator trả timeout cùng danh sách trống. Đây là điểm mạnh hơn đoạn `try/except` chỉ bắt TimeoutError mà không hề thiết lập thời hạn.

Tính tương thích Python cần chú ý: ở một số phiên bản, `asyncio.TimeoutError` và built-in `TimeoutError` không hoàn toàn cùng lớp. Mã bắt cả hai; test timeout đã được thực thi trong môi trường hiện tại. Trường status của trace được khởi tạo trước try để ngay cả lỗi vẫn ghi nhận được, tránh lỗi logging che mất lỗi gốc.

Deadline ở mức coroutine không tương đương khả năng dừng an toàn mọi thư viện blocking hoặc một tiến trình ngoài. Với HTTP, cần timeout của client; với CPU job, có thể cần process riêng; với thao tác ghi, timeout còn để lại trạng thái không biết đã hoàn tất chưa. Prototype này chỉ đọc dữ liệu nên không có retry giao dịch hoặc duplicate purchase.

Các chỉ số theo dõi gồm success, violation, P50/P95 latency và số message. Một request–response được tính hai message. Trace hiện lưu task ID và thời lượng, không chứa khóa API hay dữ liệu cá nhân. Cấu trúc này đủ để minh họa quan sát vận hành mà chưa phải hệ tracing phân tán đầy đủ.

13 Unit 12 — Case study thương mại điện tử

Thiết kế hoàn chỉnh có thể thêm intent, preference, catalog, ranker, inventory và policy. Tuy nhiên, số lượng vai trò nên tăng khi có trách nhiệm rõ. Với dữ liệu hiện tại, preference chưa có nguồn người dùng, vì vậy không tạo hồ sơ giả rồi gọi là cá nhân hóa. Score giá tăng là một baseline minh bạch.

Một bộ dữ liệu thật có thể đánh giá retrieval Recall@k, ranking NDCG@k và tỷ lệ vi phạm điều kiện. Các chỉ số này cần relevance label hoặc ground truth phù hợp. Bộ benchmark hiện dùng ID mong đợi và status, không báo NDCG vì chưa có nhãn graded relevance. Phân biệt chỉ số chưa đo giúp báo cáo giữ đúng phạm vi bằng chứng.

Gợi ý triển khai tiếp: thêm catalog adapter chỉ đọc, snapshot có version và stock TTL theo loại hàng. Trước khi đánh giá, đóng băng tập yêu cầu và catalog snapshot để mọi kiến trúc thấy cùng dữ liệu. Khi dùng thông tin online, cần ghi timestamp để biết sai do mô hình hay do dữ liệu thay đổi.

14 Unit 13 — Case study du lịch

Một tourism MAS có thể gồm preference, attraction, route và budget agent. Trạng thái lịch trình chứa điểm đã chọn, thời điểm đến, thời gian tham quan, chi phí và ràng buộc. RouteAgent phải tính cả di chuyển, không chỉ cộng thời lượng tham quan. BudgetAgent phải bao gồm phí và vận chuyển, không chỉ vé vào cửa.

Ví dụ tổng hợp: người dùng có 6 giờ và ngân sách 500 đơn vị; ba điểm cần lần lượt 120, 90, 150 phút; tổng di chuyển 80 phút. Chọn cả ba cần 440 phút, vượt 360 phút dù từng điểm riêng lẻ đều phù hợp. Coordinator phải bỏ điểm hoặc xin thay đổi ưu tiên mềm, không tạo lịch trình bất khả thi.

Bộ mã hiện không triển khai tourism planner và không dùng thông tin giờ mở cửa thực tế. Đây là bài thiết kế chuyển miền hoàn chỉnh về vai trò và ràng buộc, phù hợp với Unit 13; phần thực thi năm exercise được chọn ở e-commerce. Nếu phát triển planner thật, dữ liệu mở cửa và thời gian di chuyển phải lấy từ nguồn hiện hành có thể kiểm tra.

15 Unit 14 — Case study hỗ trợ hành chính y tế

Phạm vi phù hợp cho prototype sinh viên là chuẩn bị giấy tờ, hướng dẫn quầy tiếp nhận và hỗ trợ lịch hẹn. IntakeAgent lấy yêu cầu; InformationAgent chỉ đọc nguồn đã duyệt; SchedulingAgent đọc lịch; PreparationAgent tạo danh sách việc cần chuẩn bị; Coordinator quản lý quyền và xác nhận thao tác nếu có thay đổi lịch thật.

Bài này không triển khai phân loại cấp cứu, chẩn đoán hoặc điều trị. Lớp hỗ trợ hành chính y tế trong classifier chỉ là nhãn tác vụ hành chính tổng hợp. Do vậy accuracy của lớp này không được gọi là độ chính xác y khoa. Từ "healthcare" trong một mô hình routing không tự mang ý nghĩa clinical validation.

Đánh giá hành chính nên kiểm tính đúng thông tin, độ đầy đủ và bảo vệ dữ liệu. Khi không có nguồn xác nhận, hệ phải trình bày chưa biết. Cách xử lý inventory_unverified ở e-commerce minh họa nguyên tắc tương tự, nhưng không đủ để chứng nhận an toàn cho miền y tế.

16 Unit 15 — So sánh ba miền và quyền sở hữu ràng buộc

Miền	Dữ liệu có thẩm quyền	Ràng buộc chính	Khi thiếu dữ liệu
E-commerce	Catalog, price, inventory	Budget, ID hợp lệ, stock mới	Không khẳng định availability
Tourism	Điểm đến, giờ mở cửa, thời gian đi	Khả thi thời gian, budget	Trả phương án chưa xác minh hoặc sửa lịch
Healthcare administration	Tài liệu cơ sở và lịch hẹn	Đúng thông tin, quyền, tối thiểu dữ liệu	Dừng thao tác và chuyển kiểm tra phù hợp

Ở cả ba miền, mỗi ràng buộc cần người sở hữu. Nếu "ai cũng kiểm" nhưng không có hợp đồng, có thể không thành viên nào thực hiện đầy đủ. Coordinator nên có một checklist kiểm được bằng máy cho điều kiện bắt buộc, còn các đánh giá mềm để agent chuyên môn cung cấp.

Phân biệt hành động đề xuất với hành động ghi là cần thiết khi chuyển prototype sang ứng dụng. Trong bài hiện tại, mọi actuator chỉ trả kết quả nên có thể chạy tự động. Nếu thêm đặt hàng hoặc thay lịch, phải thiết kế thao tác có khả năng xem trước và xác nhận rõ, cùng idempotency và kiểm tra kết quả dịch vụ.

17 Unit 16 — Thiết kế benchmark có đối chứng

B0 là pipeline một thành phần có cùng kiểm chứng factual. B1 là MAS theo luật có coordinator validation. B2 thêm classifier intent trước MAS. B3 tắt policy/freshness validation để đo ảnh hưởng của việc bỏ kiểm. Đây là bốn cấu hình có mục đích khác nhau; B3 là ablation chẩn đoán, không phải một sản phẩm được đề xuất sử dụng.

Tập đánh giá có 11 ca, mỗi ca lặp 20 lần, tổng 220 yêu cầu mỗi kiến trúc. Các ca gồm bình thường, query không biết, query rỗng, budget 0, hết hàng, budget âm, timeout, malformed, stale và forged. Repetition dùng để ước lượng latency cục bộ; nó không biến 11 ca thành 220 tác vụ độc lập về nội dung.

Success yêu cầu đồng thời status và danh sách ID bằng ground truth. Violation kiểm item ngoài catalog, vượt budget hoặc khẳng định availability từ dữ liệu stale. B0/B1/B2 đạt đủ các ca; B3 chỉ đạt 8/11 loại ca và có vi phạm. Số đo cụ thể được chèn từ CSV ở phần kết quả, không viết tay.

Có hai giới hạn so sánh quan trọng. B0 mô phỏng trạng thái timeout đồng bộ nên không chờ deadline thật; không thể dùng P95 của nó để kết luận một hệ dịch vụ đơn luôn nhanh hơn MAS cùng tỷ lệ đó. B2 dùng cùng một câu mua laptop để định tuyến trước mọi shopping case; benchmark này đo overhead tích hợp classifier, **không** đo routing đa miền. Chất lượng routing được đánh giá riêng trên tập 108 câu và OOD.

18 Unit 17 — Hoàn thành năm bài thực hành

18.1 Exercise 1: Đặc tả tác tử và PEAS

Ứng dụng chọn là shopping assistant. Bảng đặc tả ở Unit 3 có bốn worker và coordinator, đầy đủ input, output, quyền và tiêu chí thành công. PEAS bao gồm điều kiện từ chối đúng, không chỉ số lượng sản phẩm. Sơ đồ Unit 5 thể hiện nguồn quyết định và thứ tự truyền dữ liệu. Đó là bằng chứng cho yêu cầu tối thiểu ba agent.

18.2 Exercise 2: Mở rộng shopping system

Tệp `slide_baseline.py` chép lại luồng bốn sản phẩm từ Unit 7–9 và đã chạy năm kiểm tra: truy hồi tag, không vượt budget, unknown query, empty query và chỉ trả candidate. Output

được lưu trong `slide_baseline.json`. Từ baseline hai agent đó, bản mở rộng bổ sung inventory và policy; B0 trong benchmark là một baseline đơn có cùng điều kiện kiểm chứng để so công bằng, khác bản slide ban đầu. Policy kiểm ID nằm trong catalog/candidate và không vượt budget; inventory kiểm stock; coordinator kiểm response envelope và TTL. Kịch bản portable/100 chứng minh không trả laptop vượt ngân sách, còn sports/100 chứng minh không trả sản phẩm stock 0.

Để tự mở rộng, thêm PreferenceAgent nhận danh sách ID cùng sở thích có schema. Hãy giữ budget ở lớp filter và dùng sở thích chỉ xếp hạng trong miền hợp lệ. Viết test rằng một sản phẩm có score cao nhưng vượt budget vẫn không xuất hiện. Đây là cách kiểm tra integration, thay vì chỉ thử riêng hàm chấm điểm.

18.3 Exercise 3: Huấn luyện intent classifier

`train_intent.py` tạo dữ liệu, chia nhóm, huấn luyện, chọn checkpoint, chọn threshold và xuất per-class metrics. `intent_model.pt` chứa state_dict, vocabulary và threshold. Không tải một checkpoint không rõ nguồn. Sau chạy, xem confusion matrix, loss train/validation và các câu OOD, rồi giải thích vì sao kết quả 100% vẫn không đủ cho triển khai.

Bài đã có accuracy, precision, recall và F1 từng lớp trên test, đúng yêu cầu của slide. Phần confidence threshold là cơ chế abstention có thể chạy, đồng thời báo cáo lỗi false acceptance. Muốn cải thiện nên tạo tập dữ liệu phong phú hơn và giữ test độc lập sau khi chỉnh quy tắc.

18.4 Exercise 4: Failure injection

Bốn lỗi bắt buộc là timeout, empty result, malformed message và stale availability đều có code và test. Bổ sung forged recommendation, sai correlation ID, budget NaN và budget âm. Người học có thể chạy `test_tutorial.py` để xem ca nào thất bại khi cố ý bỏ validation.

Ví dụ thực hành: đổi `Coordinator(validate=False)` rồi chạy stale và forged. Kết quả vẫn có thể mang status ok nhưng vi phạm factual. Sau đó bật validation, quan sát availability_unverified hoặc contract_error. Bài tập cho thấy đánh giá an toàn phải xét nội dung và provenance, không chỉ việc hàm trả về mà không ném exception.

18.5 Exercise 5: So sánh kiến trúc

Bảng benchmark đi kèm cho thấy B0 và B1 có cùng tỷ lệ success trên bộ ca. Vì vậy kết quả **không chứng minh chia nhiều agent tăng chất lượng** ở catalog toy. B1 có thêm message và latency; lợi ích là ranh giới trách nhiệm, trace và khả năng thay thành phần. B3 cho thấy bỏ kiểm chứng làm giảm chất lượng dù vẫn có nhiều agent.

Một so sánh tiếp theo nên tăng quy mô dữ liệu và độc lập nguồn lỗi, giữ cùng tác vụ, lặp theo nhiều tải và dùng cùng cơ chế timeout cho cả B0/B1. Nếu muốn chứng minh lợi ích chuyên môn hóa, cần tác vụ mà các agent có dữ liệu hoặc công cụ khác nhau thực sự, và baseline đơn được cấp nguồn lực tương xứng.

19 Unit 18 — Kiểm thử, rubric và lỗi thường gặp

Có 17 test tự động cho tutorial, bao gồm đường bình thường, ngân sách, empty/unknown, hết hàng, input sai, timeout, message malformed, stale, forged, correlation, trace, chia nhóm dữ liệu, OOD đơn giản và tương đương baseline. Test cuối đối chiếu 20 tổ hợp query/budget bình thường. Các test lỗi kiểm hành vi tại ranh giới, không chỉ gọi lại công thức trong implementation.

Rubric của slide được đối chiếu bằng sản phẩm: PEAS và vai trò có trong báo cáo; nguồn Python và README có sẵn; timeout thực và validation có test; classifier đã huấn luyện; benchmark có dữ liệu thô; phạm vi synthetic được ghi rõ; demo và slide trình bày có kèm. Các yêu cầu này được gắn vào manifest để người đọc kiểm tra nhanh.

Những lỗi cần tránh: gọi mạng chưa train là predictor hợp lệ; dùng cùng tập để chọn ngưỡng và báo test; lấy output giả làm bằng chứng chạy; gọi nhiều hàm là chuyên gia mà không có trách nhiệm; xem response ok là thành công dù có ID bịa; và suy ra readiness triển khai từ vài dữ liệu tổng hợp.

20 Unit 19 — Kiến trúc tham chiếu và hướng phát triển

Nếu mở rộng thành dịch vụ, nên thêm adapter dữ liệu có version, cấu hình timeout theo loại công cụ, retry cho thao tác đọc có giới hạn và logging không chứa thông tin nhạy cảm. Những thao tác ghi cần idempotency key, đọc lại xác nhận và mô hình trạng thái giao dịch. Không nên tự thêm một vòng retry vô hạn khi không hiểu nguyên nhân lỗi.

Nếu mở rộng learned component, nên bắt đầu bằng ranking dataset có nhãn và đánh giá riêng retrieval/ranking. Giữ invariant kiểm được bằng máy ở coordinator. Nếu thêm LLM, coi output là proposal và giữ validator; lưu model/prompt version; đo số token, độ trễ, groundedness và tỷ lệ lỗi theo nhóm yêu cầu.

Đối với dữ liệu intent, bước cải tiến ưu tiên là dữ liệu tự nhiên, từ vựng chưa thấy, câu đa ý định, phủ định, thiếu thông tin và ngoài miền. So sánh classifier với keyword baseline trên cùng dữ liệu. Không nên tăng mạng sâu trước khi kiểm tra bộ dữ liệu có thực sự yêu cầu năng lực đó hay chưa.

21 Unit 20 — Tổng kết, demo và câu hỏi bảo vệ

Một bản demo 5–7 phút có thể đi qua sáu bước: giới thiệu PEAS; chạy portable/100; mở trace; thử timeout và stale; trình bày confusion matrix cùng lỗi OOD; kết thúc bằng bảng B0–B3. Đây là trình tự cho thấy cả cơ chế và giới hạn, không chỉ một output thành công được chọn sẵn.

Câu hỏi bảo vệ gợi ý: vì sao dùng MAS ở catalog nhỏ; timeout nào thực sự được áp dụng; vì sao softmax cao vẫn sai; classifier có phải RL không; B3 đo điều gì; vì sao baseline đơn vẫn cần kiểm kho; và dữ liệu nào sẽ cần thêm trước khi dùng thật. Các câu trả lời phải gắn với code, dữ liệu và protocol đã công bố.

Thông điệp kỹ thuật của tutorial là xây một hệ có hợp đồng, kiểm chứng và bằng chứng đo được. Phần học máy đóng góp một thành phần dự đoán đã train và có giới hạn được đo; phần điều phối giữ các điều kiện quan trọng bất kể dự đoán. Cả hai cần được kiểm tra bằng cùng tiêu chí nhiệm vụ.

22 Kết quả thực hành và bằng chứng chạy

22.1 Kiến trúc B0–B3

Các số dưới được lấy từ mas_benchmark.csv. Mỗi hệ chạy 11 ca, lặp 20 lần. Success có tính cả trạng thái từ chối đúng; tỷ lệ không đồng nghĩa tỷ lệ yêu cầu có gợi ý sản phẩm.

Hệ	Success (%)	P50 ms	P95 ms	Message/request	Violation (%)
B0_single	100.00	0.0048	0.0090	0.00	0.00
B1_rule_MAS	100.00	0.2406	11.0973	6.00	0.00
B2_learned_MAS	100.00	0.3104	11.4029	6.00	0.00
B3_no_validation	72.73	0.2172	11.3042	4.73	18.18

Không so tỷ lệ tốc độ B0/B1 trên các ca lỗi: baseline đơn trả trạng thái timeout được tiêm đồng bộ, còn MAS chờ deadline thực. Phần learned routing trong benchmark luôn nhận một câu ecommerce cố định, nên chỉ phản ánh overhead tích hợp. Đánh giá đa miền nằm ở bảng classifier riêng.

22.2 Chỉ số classifier trên held-out test

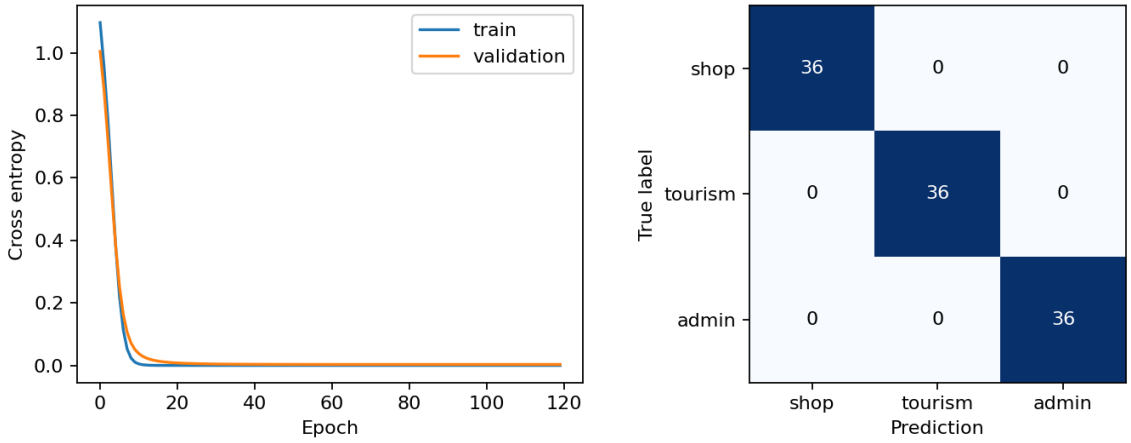
Lớp	Precision	Recall	F1	Số câu
ecommerce	1.0000	1.0000	1.0000	36
tourism	1.0000	1.0000	1.0000	36
healthcare admin	1.0000	1.0000	1.0000	36

Accuracy: 1.0000; macro-F1: 1.0000; threshold 0.5; coverage 1.0000. Tập train, validation và test: 270, 54 và 108 câu. Vocabulary 79; seed 42.

22.3 Ngoài miền: những lỗi cần giữ lại để phân tích

Câu thử	Kết quả	Confidence
xin chào	clarify	0.4943
thời tiết hôm nay thế nào	ecommerce	0.9877
viết thơ cho tôi	clarify	0.6866
giải phương trình	tourism	0.9311
cảm ơn bạn	clarify	0.4943
abc xyz	clarify	0.4943

Có 4/6 câu được từ chối và 2/6 bị nhận nhầm vào miền dù confidence cao. Sáu câu là probe nhỏ, không phải một benchmark OOD đại diện. Không dùng tỷ lệ này để ước lượng lỗi trên mọi yêu cầu người dùng.



Hình 2: Loss train/validation và confusion matrix trên 108 câu test tổng hợp.

22.4 Kết quả failure injection và test

Ca lỗi	Status của B1	Số item
0	ok	2
1	no_results	0
2	no_results	0
3	no_results	0
4	no_results	0
5	ok	1
6	timeout	0
7	contract_error	0
8	availability_unverified	0
9	contract_error	0
10	invalid_input	0

Có 17 test tutorial chạy thành công; log đầy đủ nằm trong build/tutorial_tests.log. Hệ đã kiểm 4 lỗi bắt buộc của slide và thêm input NaN, ID giả, sai correlation. Trường hợp OOD trong test chỉ xác nhận câu không có token miền trả clarify, không phủ nhận các lỗi OOD đã báo ở bảng trên.

```
python3 -m unittest discover -s tutorial -p 'test_*.py'
.................
Ran 17 tests
OK
```

22.5 Mã thực thi cần đọc khi bảo vệ

Listing 1: Deadline, xác thực envelope và trace của coordinator

```
class Coordinator:
    def __init__(self,validate=True,deadline=.01):
        self.catalog=CatalogAgent();self.recommender=RecommendationAgent();self.inventory=InventoryAgent()
        self.policy=PolicyAgent();self.validate=validate;self.deadline=deadline;self.trace=[]
    async def call(self,agent,m,expected):
        start=time.perf_counter(); status='error'
        try:
            result=await asyncio.wait_for(agent.handle(m),timeout=self.deadline)
            if not isinstance(result,Message):raise TypeError('Malformed response')
            if result.task_id!=m.task_id or result.receiver!=m.sender or result.sender!=agent.name or result
    .kind!=expected:
                raise ValueError('Response correlation/type violation')
            status='ok';return result
        except (TimeoutError,asyncio.TimeoutError,ValueError,TypeError,KeyError):
            status='error';raise
        finally:
            self.trace.append({'task_id':m.task_id,'agent':agent.name,'kind':m.kind,
                'status':status,'milliseconds':(time.perf_counter()-start)*1000})
```

Listing 2: Luồng điều phối và trạng thái kết thúc

```
    async def shop(self,query,budget,scenario='normal',task_id='demo'):
        self.trace=[]
        if not isinstance(query,str) or not isinstance(budget,(int,float)) or not math.isfinite(budget) or
    budget<0:
            return {'status':'invalid_input','items':[]}
        def msg(receiver,kind,payload):return Message(task_id,'coordinator',receiver,kind,payload)
        try:
            c=await self.call(self.catalog,msg('catalog','SEARCH',{'query':query,'scenario':scenario}),'
    CANDIDATES')
            candidates=c.payload['ids']
            if not candidates:return {'status':'no_results','items':[]}
            r=await self.call(self.recommender,msg('recommender','RANK',{'ids':candidates,'budget':budget})
    ,'RANKED')
            ids=r.payload['ids']
            # Deliberate adversarial component output, to test coordinator validation.
            if scenario=='forged':ids=['p99','p1']+ids
            if self.validate:
                p=await self.call(self.policy,msg('policy','VALIDATE',{'ids':ids,'candidates':candidates,'
    budget':budget}),'VALIDATED')
                ids=p.payload['ids']
            inv=await self.call(self.inventory,msg('inventory','CHECK',{'ids':ids,'scenario':scenario}),'
    AVAILABILITY')
            if self.validate and (not isinstance(inv.payload.get('age_seconds'),(int,float)) or not 0<=inv.
    payload['age_seconds']<=60):
                return {'status':'availability_unverified','items':[]}
            stocks=inv.payload['stock']
            if self.validate and (set(stocks)!=set(ids) or any(type(n) is not int or n<0 for n in stocks.
    values())):
                raise ValueError('Invalid stock schema')
            lookup={p.id:p for p in PRODUCTS}
            items=[{'id':i,'price':lookup[i].price if i in lookup else 0,'availability':'verified'}
                   for i in ids if stocks.get(i,0)>0 or not self.validate]
            return {'status':'ok' if items else 'no_results','items':items}
        except (TimeoutError,asyncio.TimeoutError):return {'status':'timeout','items':[]}
        except (ValueError,TypeError,KeyError):return {'status':'contract_error','items':[]}
```

Đoạn mã đầy đủ trong tutorial/mas_system.py là nguồn sự thật khi cần kiểm các dòng sau trích đoạn. Notebook thể hiện lời gọi trực tiếp, đầu ra thực tế, dữ liệu học và các bảng kết quả; bản HTML cho phép xem offline mà không cần chạy kernel.

Tài liệu tham khảo

[1] PGS.TS. Trần Đình Quế (tài liệu môn học được cung cấp), *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*, slide_multiAgent_system_20.10.2026.pdf, 25 trang, 2026. Nguồn cục bộ trong đề bài.

[2] *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, Intelligent System Development, 2026–2027, assignment_Agent02_due_29.10.pdf, 13 trang; agent02_code.tex và agent02_complementary_code.pdf. Tài liệu cục bộ do người dùng cung cấp.

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*, arXiv:2501.06322v1, 10/01/2025. https://arxiv.org/abs/2501.06322v1. Phân tích trực tiếp bản PDF 35 trang được cung cấp.

[4] M. Naeem, S. T. H. Rizvi, A. Coronato, *A Gentle Introduction to Reinforcement Learning and Its Application in Different Fields*, IEEE Access, vol. 8, pp. 209320–209344, 2020. DOI: 10.1109/ACCESS.2020.3038605. Phân tích trực tiếp bản PDF 25 trang được cung cấp.

[5] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. Tài liệu nền tảng được cả slide và paper RL dẫn chiếu.

[6] C. J. C. H. Watkins, P. Dayan, *Q-learning*, Machine Learning 8, 279–292, 1992. DOI: 10.1007/BF00992698.

[7] Q. Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*, 2023. https://arxiv.org/abs/2308.08155.

[8] S. Hong et al., *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*, 2023, bản cập nhật 2024. https://arxiv.org/abs/2308.00352.

[9] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, *Improving Factuality and Reasoning in Language Models through Multiagent Debate*, 2023. https://arxiv.org/abs/2305.14325.

[10] PyTorch, *CrossEntropyLoss*, tài liệu chính thức. https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html. Đối chiếu giao diện ngày 28/10/2026; môi trường thực nghiệm dùng PyTorch 2.8.0.

[11] Python documentation, *Coroutines and Tasks: asyncio.wait_for*. https://docs.python.org/3/library/asyncio-task.html. Phần timeout được kiểm trực tiếp bằng tests trong môi trường thực thi.

[12] Bộ mã và dữ liệu tự thực thi của báo cáo: tutorial/, assignment/, results/, build/; seed và cấu hình lưu trong summary.json, intent_metrics.json và mas_metrics.json, ngày 28/10/2026. Đây là nguồn kết quả mô phỏng, không phải nguồn dữ liệu thị trường thực tế.